Database
(SOC-3020)

Alessandro Agostini[1,2]

aagostini@inha.ac.kr
agostini@disi.unitn.it

[1]LDKR-Group, School of Computer and Information Engineering
Inha University in Tashkent (IUT), Uzbekistan

[2]DISI, University of Trento, Italy

WEEK 2





DATABASE

The Relational Model

WEEK 2

Code: W2.L1L2

TODAY'S AGENDA

1. From Previous Week
2. The Relational Model
3. Data and Data Relationships
4. Data Semantics
5. Data Constraints: Entity Integrity
6. Data Constraints: Referential Integrity
7. Relational Operations
8. Appendix: MySQL - Installation Guidelines



Any question from you?

...



Remember: DBMS = Data + Programs

- Remember: a database (management) system is a "sum" of
  - **interrelated data** (Abstract World) and a
  - **set of programs** that allow users to access and modify (*conveniently, efficiently*) these data.
- A purpose of a database (management) system is to provide each user with an *abstract* view of the data.
- Such view is provided by a specific **data model**.





Remember: Abstract Views on Data

- **Physical level:** describes how a record (e.g., instructor) is physically stored.
- **Logical level:** describes what data are stored in database, and the relationships among the data.

Example:

type *instructor* = **record**

logical association; i.e., we transform data into information (about instructors) Information is defined from ***structured data*** (the record)

ID : string;
name : string;
dept_name : string;
salary : integer;

} *data ("cells")*

end;

- **View level:** application programs hide details of some data types. Views used to hide some information ("logical level") for security/privacy purposes. (E.g.: an employee's salary)





Data Dictionary (or: System Catalog)

- Stores **metadata,** that is, data *about* data, such as (for the Relational Model) data about relations:
  - names of relations
  - names, types and lengths of attributes of each relation
  - names and definitions of views
  - integrity constraints
- User and accounting information, e.g., passwords
- Statistical and descriptive data
  - number of tuples and columns in each relation
- Physical file organization information
  - How relation is stored (sequential/hash/…) /Physical location
- Information about indices

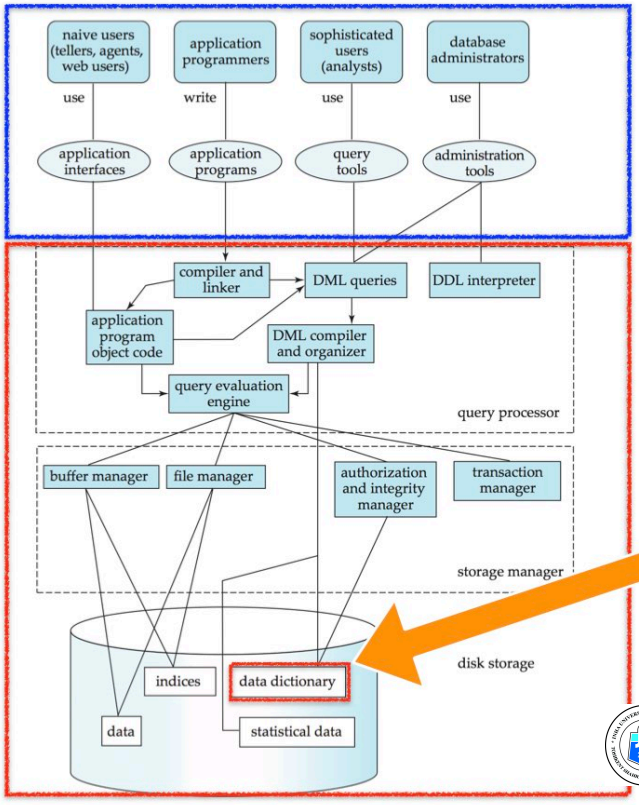




DBMS Architecture

users and administrators

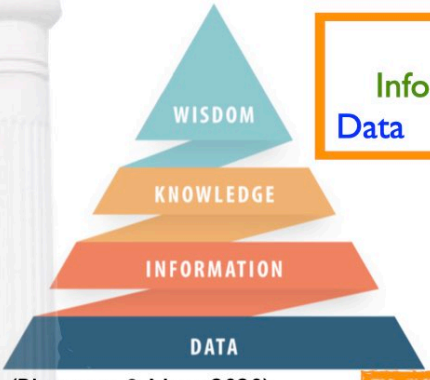
internal components





Data Models

The **World** is…
"the totality of Facts" (Wittgenstein)
i.e.,
.
.
.
Data!



(Blumauer & Nagy, 2020)

Knowledge
Information
Data

Real World
(Mini World)

- Data and Data relationships
- Data semantics
- Data constraints (*internal*, business rules, …)



Relational Data Model

Knowledge
Information
Data

Real World
(Mini World)

Data!

- **Data and Data relationships**
- **Data semantics**
- **Data constraints**

Week's focus

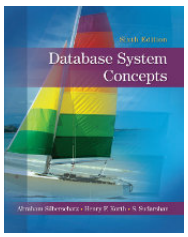


Outline

1. From Previous Week
2. The Relational Model
3. Data and Data Relationships
4. Data Semantics
5. Data Constraints: Entity Integrity
6. Data Constraints: Referential Integrity
7. Relational Operations
8. Appendix: MySQL - Installation Guidelines

The Relational (Data) Model

Ch 2, T; sections 2.1-2.4



Main Reference (history)

Suggested reading (non-technical parts):

```
@Article{Codd70cacm,
  author      = "E. F. Codd",
  title       = "A Relational Model of Data for Large Shared Data Banks",
  journal     = "Communications of the ACM",
  year        = "1970",
  volume      = "13",
  number      = "6",
  pages       = "377--387"
}
```





Information Retrieval P. BAXENDALE, Editor

A Relational Model of Data for Large Shared Data Banks

E. F. Codd
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models are discussed. A model based on *n*-ary relations, a normal form for data base relations, and the concept of a universal data sublanguage are introduced. In Section 2, certain operations on relations (other than logical inference) are discussed and applied to the problems of redundancy and consistency in the user's model.

KEY WORDS AND PHRASES: data bank, data base, data structure, data organization, hierarchies of data, networks of data, relations, derivability, redundancy, consistency, composition, join, retrieval language, predicate calculus, security, data integrity
CR CATEGORIES: 3.70, 3.73, 3.75, 4.20, 4.22, 4.29

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for noninferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the "connection trap").

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted data systems, and also the relative merits (from a logical standpoint) of competing representations of data within a single system. Examples of this clearer perspective are cited in various parts of this paper. Implementations of systems to support the relational model are not discussed.

1.2. Data Dependencies in Present Systems

The provision of data description tables in recently developed information systems represents a major advance toward the goal of data independence [5, 6, 7]. Such tables facilitate changing certain characteristics of the data representation stored in a data bank. However, the variety of data representation characteristics which can be changed *without logically impairing some application programs* is still quite limited. Further, the model of data with which users interact is still cluttered with representational prop-



Relational Model

- Describes data at logical level (and view level (*)).
  - (*) The view level *abstracts* from the logical level.
- Still the primary data model for commercial data-processing nowadays:
  1. It is conceptually simple.
  2. Programmers can design it easier than previous models.
- Substantial theory exists for it (cfr. Ch. 6)



Outline

1. From Previous Week
2. The Relational Model
3. Data and Data Relationships
4. Data Semantics
5. Data Constraints: Entity Integrity
6. Data Constraints: Referential Integrity
7. Relational Operations
8. Appendix: MySQL - Installation Guidelines

Data and Data Relationships

or: How do we structure data by the Model?

- **Data and Data relationships**
- **Data semantics**
- **Data constraints** (for business rules)

Tables, or: *Relations*

- **Def**: A relational database (RDB) consists of a collection of **tables** (also called relations), each of which is assigned a unique name.
- **Def**: A table is a set of rows (or "tuples"), each row representing a relationship R among data in the table's columns (or "attributes").
- Formally: $Table_R = \{<d_1,\ldots d_n>|\ R(d_1,\ldots d_n)\}$, or $Table_R(a1,\ldots,an) = \{<d_1,\ldots d_n>|\ R(a_1/d_1,\ldots,a_n/d_n)\}$



Example 1: *instructor*

instructor:

attributes (or columns)

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

tuples (or rows)



Example 2: *instructor*

instructor:

attributes (or columns)

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

tuples (or rows)





Tables (*cont.*)

- **Remark 1**: The order in which tuples appear in a relation is irrelevant, since a relation is a *set* of tuples. Again, remember:

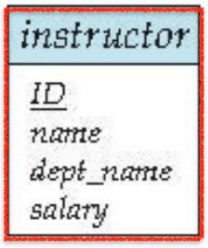
set!

$$Table_R = \{<d_1,\ldots d_n> | R(d_1,\ldots d_n)\}$$

- **Def**: A relation instance is the content of the relation, i.e., the values of all the relation attributes, at a given point in time.



Schema / Instance (I)



Def: A relation schema is the logical view of the relation. (*So, it's its "design", or "structure".*)

- **Example**: for relation "*student*", the schema is:

(*ID,name,dept_name,tot_cred*)

Note: notation may vary; cf. graphical notation of tables.

Then, to represent students we write:

student (*ID,name,dept_name,tot_cred*)

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Def: A relation instance is the value of the relation'attributes at a given point in time.



Example: *course*

course:

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4



Schema / Instance (II)

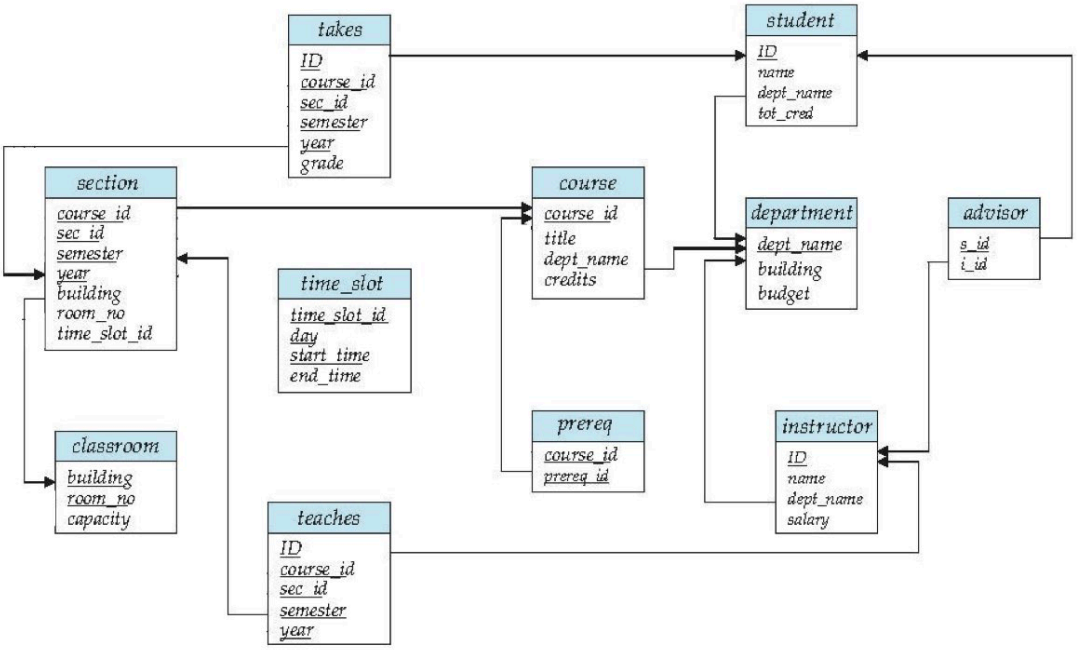
- **Def**: The schema of a database is the logical/ conceptual view on it; say, its logical design.
- **Def**: An instance of a database is the content of all its relations, i.e., the values of all relations' attributes *at a given point in time*.

- Concepts of "schema" and "instance" are as "variable type" / "variable value" in programming.





Schema Diagram for University Database





Textual Notation

- Let's write some of the relation schemas we would need to design a RDB *for* an university, i.e:
  - *for students: student (ID,name,dept_name,tot_cred)*
  - *for instructors: instructor (ID,name,dept_name,courses)*
  - *for department: department (dept_name,building,budget)*
  - *for courses: course (course_id,title,dept_name,credits)*
  - *for courses' prerequisites: prereq (course_id,prereq_id)*
  - *for teachers: teaches (ID,course_id,sec_id,semester,year)*

  …



Knowledge in RDBMSs

- **Question**: "What students have registered to the course with course_ID = SOC3020?"
- **Answer**: Knowledge emerges from DB' schemas:
  1. *department* (*dept_name*,*course_id*)
  2. *student* (*ID*,*name*,*dept_name*,*tot_cred*)
  - From 1 we can find the *dept_name* of all the departments that offer the course SOC3020.
  - From 2 we find information of students in *dept_name*.



Outline

1 From Previous Week

2 The Relational Model

3 Data and Data Relationships

4 Data Semantics

5 Data Constraints: Entity Integrity

6 Data Constraints: Referential Integrity

7 Relational Operations

8 Appendix: MySQL - Installation Guidelines

Data Semantics

or: How do we mean data by the Model?

- Data and Data relationships
- Data semantics
- Data constraints (for business rules)



Attributes' Domain and Type

- The set of allowed values for each attribute in a relation is called the **domain** of the attribute
- So, a table (relation R) is now formally defined as: $Table_R = \{<d_1, \ldots d_n> \in D_1 \times \ldots \times D_n \mid R(d_1, \ldots d_n)\}$ where D_i is the domain of the *i*th-attribute
- **NB**: In the Relational Model, attribute values are required to be **atomic**; that is, indivisible.
  - **Example**: no set of departments allowed as a value of attribute *dept_name* in table *instructor*.



Example: *instructor*

instructor:

atomic value of attributes

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000





Attributes' Domain and Null Value

- The special value ***null*** is a member of every domain. Used to denote
  - the value "unknown", or
  - the value does not exist.
    Example: *dept_name* in table *instructor*.

- **Remark:**
  The null value causes complications in the definition of many operations (e.g, updates)
  - Some attributes' value is fixed "**not null**" by design (entity integrity constraint).



Outline

1 From Previous Week

2 The Relational Model

3 Data and Data Relationships

4 Data Semantics

5 Data Constraints: Entity Integrity

6 Data Constraints: Referential Integrity

7 Relational Operations

8 Appendix: MySQL - Installation Guidelines

Data Constraints

or: How do we represent constraints on data (and why)?

- **Data and Data relationships**
- **Data semantics**
- **Data constraints** (technical and business rules)



Data Constraints (Ch 1, T; p. 11)

- A data model should describe **data constraints** :
  - Domain constraints.
  - Referential integrity constraints.
    E.g.: A course must be offered by a department that actually exist.
  - Assertions (or "Business Rules").
    E.g.: "Every student must take at least 5 courses".
  - Authorization.
    E.g.: "Who can read/update what"?
- Data constraints are defined by using DDL
- Output of DDL stored into Data Dictionary (or: System Catalog) as specific DB's data called metadata.



Integrity Constraints

- Integrity constraints are basically of two kinds:
  - Entity (or: "identity") integrity constraints;
  - Referential integrity constraints.



Keys for Constraints

- **Def (general, informal):**
  A key for a relation R defines a *data constraint* on data in R, i.e., a constraint on the values that R's attributes may have.



Entity ("Identity") Integrity

or: Keys for the Integrity of Tables

- **Data and Data relationships**
- **Data semantics**
- **Data constraints** (technical and business rules)

Towards Primary Keys

- We must specify how tuples in a given relation (i.e. rows in a table) differ each other.
  - **Observe**: They do differ, because of:
    $Table_R = \{<d_1,\ldots d_n> | R(d_1,\ldots d_n)\}$ *set!*
- To specify it, we use the values of the tuple, which must uniquely identify the tuple.
- No two tuples in a relation are allowed to have exactly the same value for all the attributes.



Example

- Suppose we have a student:

  student (*U1410099,Mario,CSE,12*),

  Then, usually we want that no other student, for instance:

  student (*X,Mario,CSE,12*)

  is allowed to have an ID like X = U1410099.
- Notation: *student.ID = U1410099*.



Super-Keys

- **Def**: A **superkey** for a relation R is a set of one or more attributes that, taken collectively, allow us to identify uniquely a tuple in R.
- **Example**: Suppose that in
  student (*ID,name,dept_name,tot_cred*)
  the *ID* attribute is "sufficient by design" to distinguish one *student* from another.

  Then *ID* is a superkey for *student*. But, say, {*dept_name*, *tot_cred*} is not (by common sense).



Candidate-Keys

- **Remark**: If K is a superkey for a relation, then any superset of K is a superkey.
- **Example**: *student* (*ID,name,dept_name,tot_cred*)
  ID superkey implies that also, say,
  {*ID,name*} and {*ID,dep_name*}
  are a superkey for *student* . [supersets of {*ID*}].
- Best superkeys are minimal, that is, do not have a proper subset that is a superkey.
- **Def**. Minimal superkeys are **candidate keys**.



Example

student (*ID*,*name*,*dept_name*,*tot_cred*)

- Suppose that, apart *ID*, also the combination of *name* and *dept_name* is sufficient to distinguish among members of the *student* relation.
- Thus, both {*ID*} and {*name*, *dept_name*} are superkeys and, also, candidate keys.
- Moreover, also {ID,*name*} is a superkey for *student*, but it is not a candidate key. (Why?)



Primary Keys

- **Def**: A **primary key** for a relation R is a candidate key *chosen by the database designer* to identify uniquely a tuple in the relation R.
- **Remark**: a primary key is for a relation.
  - This means that it is a property of the relation (and not of the individual tuples).
- Primary keys must be chosen with care (e.g., attributes's values don't change over time).



Example: *instructor*

instructor:

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

primary key

Usual notation:
ID (underlying)



Example: *instructor* (notation)

instructor:

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

primary key



Example: *department*

department:

dept_name	*building*	*budget*
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

primary key



Example: *section* (notation)

section:

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

primary key

***NB**: singular key, not keys!*



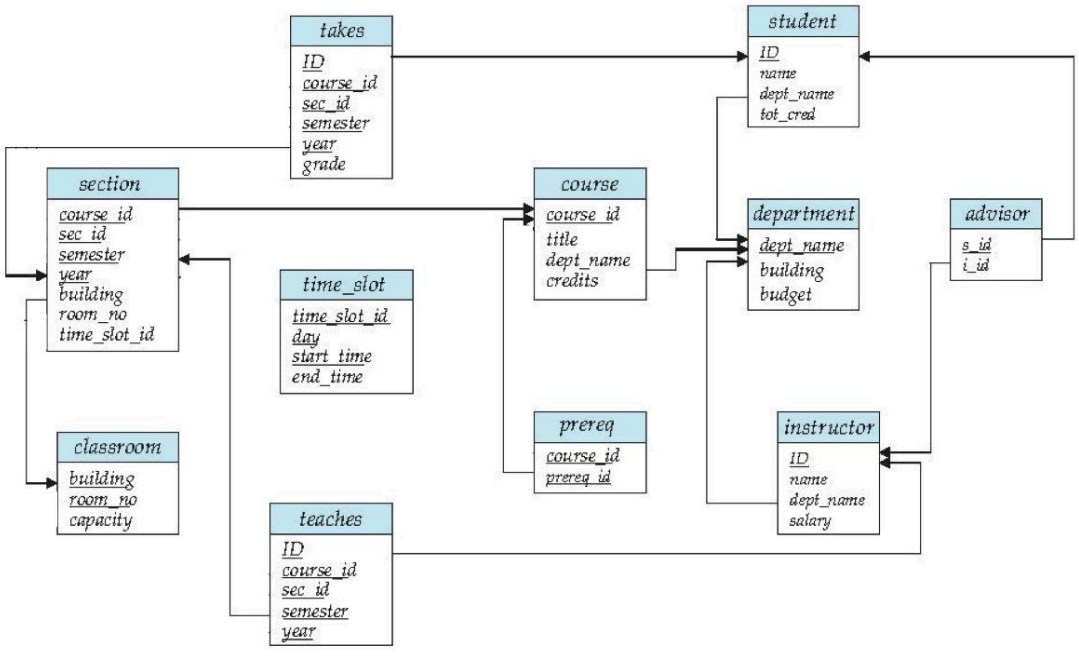
Textual Notation (*cont.*)

- Let's write some of the relation schemas and integrity constraints for our university DB, i.e:
  - *for students: student (ID,name,dept_name,tot_cred)*
  - *for instructors: instructor (ID,name,dept_name,courses)*
  - *for department: department (dept_name,building,budget)*
  - *for courses: course (course_id,title,dept_name,credits)*
  - *for teachers: teaches (ID,course_id,sec_id,semester,year)*
  - …





Schema Diagram for University Database





Outline

1 From Previous Week

2 The Relational Model

3 Data and Data Relationships

4 Data Semantics

5 Data Constraints: Entity Integrity

6 Data Constraints: Referential Integrity

7 Relational Operations

8 Appendix: MySQL - Installation Guidelines

Referential Integrity

or: Keys for the Integrity among Tables

or: How the Relational Model represents relationships...

- **Data and Data relationships**
- **Data semantics**
- **Data constraints** (technical and business rules)

Keys (*cont.*)

- **Def**: A **foreign key** from relation r_1 referencing [or: to] relation r_2 is the primary key for r_2 listed in the attributes of r_1.
- **Remark**: So, by previous definition we define a foreign (key) integrity constraint from r_1 to r_2.
  - Such constraint is: "The value of attributes in foreign key from r_1 referencing r_2 must match — if not null! — the values of attributes of primary key for r_2."



Example

instructor:

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000

department:

dept_name	*building*	*budget*
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Note: primary key for "instructor"

primary key for *department* located in *instructor* (r_1) (referencing relation of the foreign key dependency)

so, *dept_name* is a foreign key from *instructor* referencing *department*

primary key for *department* (r_2) (referenced relation of the foreign key dependency)



Foreign Key Generalized

- **Def**: A **referential (integrity) key** from relation r_2 to relation r_1 is a combination of attributes $<a_1,\ldots,a_n>$ of r_2 such that $<a_1,\ldots,a_m>$ with $n < m$ must appear in r_1.
  Note: $<a_1,\ldots,a_n>$ *not* primary key for r_1
- **Remark**: If attribute a_i is in a foreign key from r_1 referencing r_2, a **referential integrity constraint** from r_2 to r_1 requires a_i to be in r_1's attributes, as a *part of* primary key for r_1.



Example

foreign key constraint r_2 → r_1 *referential integrity constraint*

teaches:

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

section:

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
		Summer	2010	Painter	514	A
		Fall	2009	Packard	101	H
		Spring	2010	Packard	101	F
		Spring	2009	Taylor	3128	E
		Spring	2009	Taylor	3128	A
		Spring	2010	Watson	120	D
		Spring	2010	Watson	100	B
		Spring	2010	Taylor	3128	C
		Fall	2009	Taylor	3128	A
		Spring	2009	Taylor	3128	C
		Spring	2010	Packard	101	B
		Spring	2010	Painter	514	C
		Spring	2010	Packard	101	D
		Fall	2009	Watson	100	A

Note: primary key for "teaches"



Example (*cont.*)

- Suppose that we want to define a data constraint ("business rule on data") that says:

  "(1) If a section exists for a course, it must be taught by at least one instructor. However, (2) it could possibly be taught by more than one instructor."

- To enforce (1), we'd require a referential integrity constraint that says that, if a particular (*course_id, sec_id, semester, year*) instance of data appears in *section*, then the same combination must appear in *teaches*.



Example (*cont.*)

referential integrity constraint r_2

r_1

teaches:

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

section:

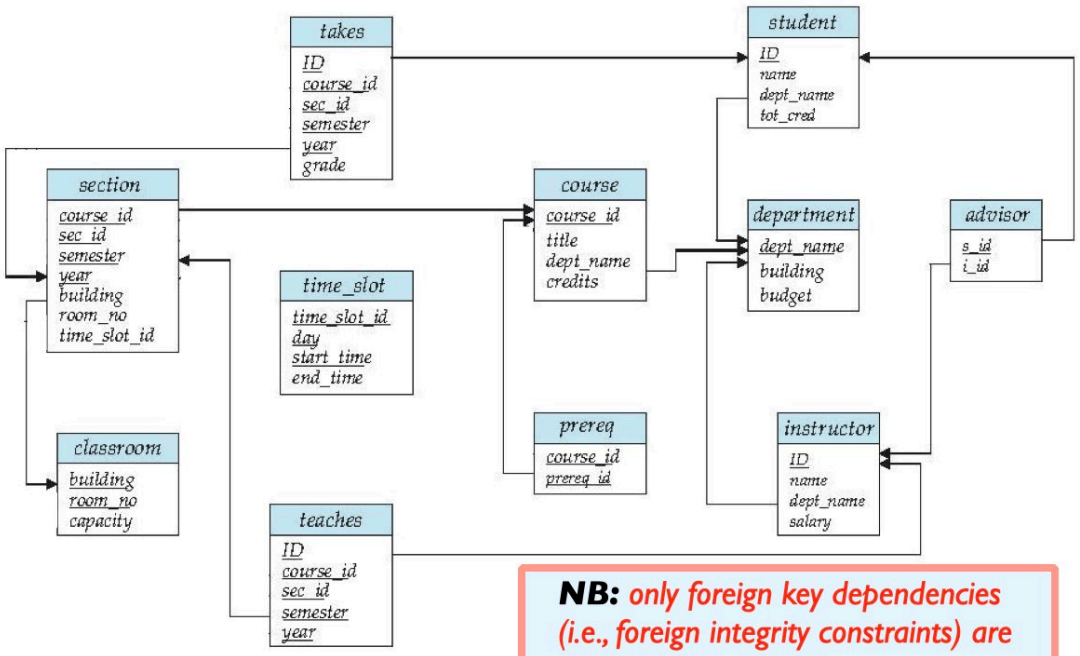
course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
		Summer	2010	Painter	514	A
		Fall	2009	Packard	101	H
		Spring	2010	Packard	101	F
		Spring	2009	Taylor	3128	E
		Spring	2009	Taylor	3128	A
		Spring	2010	Watson	120	D
		Spring	2010	Watson	100	B
		Spring	2010	Taylor	3128	C
		Fall	2009	Taylor	3128	A
		Spring	2009	Taylor	3128	C
		Spring	2010	Packard	101	B
		Spring	2010	Painter	514	C
		Spring	2010	Packard	101	D
		Fall	2009	Watson	100	A

Note: primary key for "teaches"





Schema Diagram for University Database





Outline

1 From Previous Week

2 The Relational Model

3 Data and Data Relationships

4 Data Semantics

5 Data Constraints: Entity Integrity

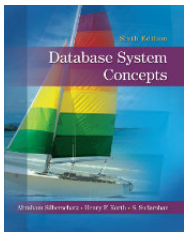
6 Data Constraints: Referential Integrity

7 Relational Operations

8 Appendix: MySQL - Installation Guidelines

Relational Operations

Ch 2, T; section 2.6



Knowledge in RDBMSs

- **Question**: "What students have registered to the course with course_ID = SOC3020?"
- **Answer**: Knowledge emerges from DB' schemas:

1. *department* (*dept_name*,*course_id*)
2. *student* (*ID*,*name*,*dept_name*,*tot_cred*)

- From 1 we can find the *dept_name* of all the departments that offer the course SOC3020.
- From 2 we find information of students in *dept_name*.





Relational Operations

- We concentrate on the following operations:
  - selection of tuples/rows,
  - projection (or: selection of attributes/columns),
  - natural join,
  - Cartesian product,
  - union,
  - set difference,
  - intersection.
- In this lecture, we'll express each operation by using the language of Relational Algebra.
  - Successively, we'll use SQL.





1. Select – selection of rows

- Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

A	B	C	D
α	α	1	7
β	β	23	10





1. Select – selection of rows

- Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

- $\sigma_{A=B \text{ and } D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10





2. Project – selection of columns

- Relation *r*

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2





2. Project – selection of columns

- Relation *r*

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

- $\prod_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2





3. Natural Join

- Relations *r*, *s*:

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

- *r* + *s*:

ID	name	salary	dept_name	building	budget
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
12121	Wu	90000	Finance	Painter	120000
15151	Mozart	40000	Music	Packard	80000
22222	Einstein	95000	Physics	Watson	70000
32343	El Said	60000	History	Painter	50000
33456	Gold	87000	Physics	Watson	70000
45565	Katz	75000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
76543	Singh	80000	Finance	Painter	120000
76766	Crick	72000	Biology	Watson	90000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000





4. Join – Cartesian Product

- Relations *r, s*:

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

- *r* x *s*:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b





5. Union of two relations

- Relations *r, s:*

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- r ∪ s:

A	B
α	1
α	2
β	1
β	3

Note: the two relations must be "similar", that is, of the same logical schema





6. Set difference of two relations

- Relations *r*, *s*:

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- *r* – *s*:

A	B
α	1
β	1

Note: the two relations must be "similar", that is, of the same logical schema





7. Intersection of two relations

- Relation *r*, *s*:

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- *r* ∩ *s*

Note: the two relations must be "similar", that is, of the same logical schema

A	B
α	2

Note: $r \cap s = r - (r - s)$



Appendix

1 From Previous Week

2 The Relational Model

3 Data and Data Relationships

4 Data Semantics

5 Data Constraints: Entity Integrity

6 Data Constraints: Referential Integrity

7 Relational Operations

8 Appendix: MySQL - Installation Guidelines

MySQL

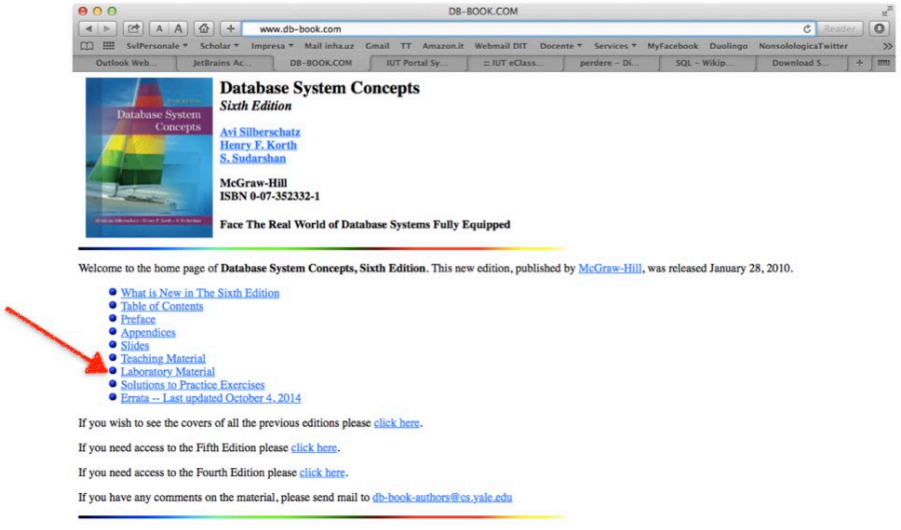
Installation Guidelines

Prepare for Practice



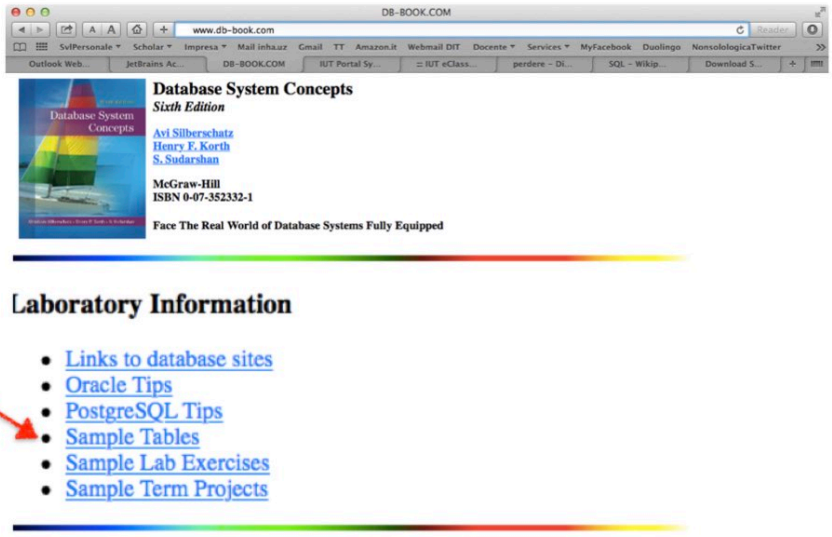
Prepare for Practice

- Textbook website: http://www.db-book.com



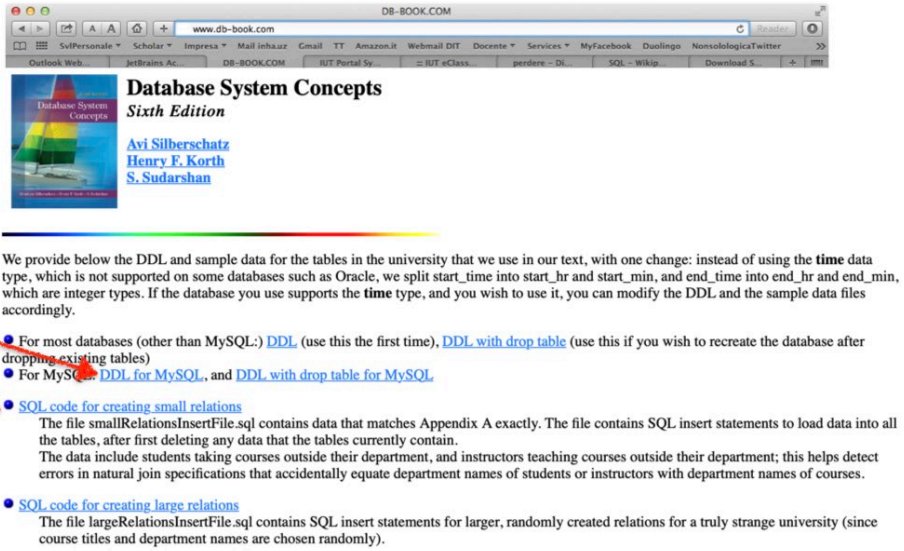
Prepare for Practice

- Textbook website: http://www.db-book.com

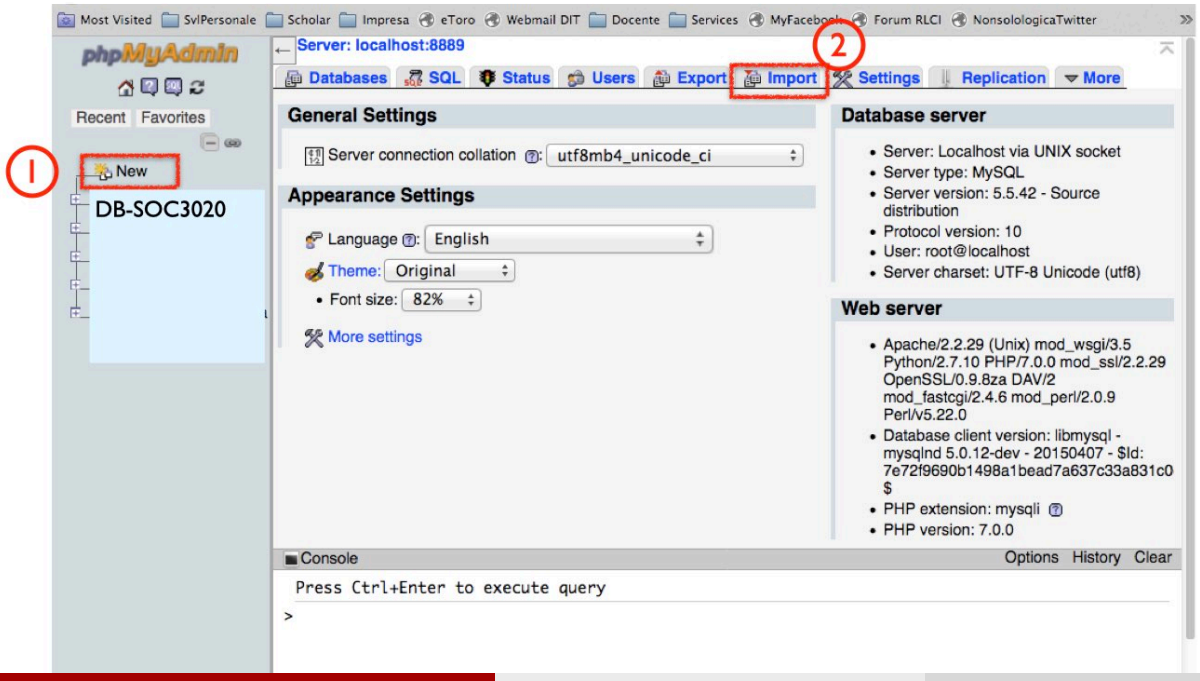


Prepare for Practice

- Textbook website: http://www.db-book.com



Prepare for Practice



Summary

1 From Previous Week

2 The Relational Model

3 Data and Data Relationships

4 Data Semantics

5 Data Constraints: Entity Integrity

6 Data Constraints: Referential Integrity

7 Relational Operations

8 Appendix: MySQL - Installation Guidelines



DATABASE

End of Slides

WEEK 2

Code: W2.L1L2